



The ASSESSMENT 2 BRIEF

MODULE CODE	COM4113
MODULE TITLE	Tech Stack
MODULE LEADER	Dr Yashar Baradaranshokouhi
ASSESSMENT TITLE	Tech Stack Product
WEIGHTING	70%

ASSESSMENT LEARNING OUTCOMES

Upon successful completion of this assessment, you will be able to:

1. Apply relevant tools and techniques to design and develop a web application, justifying the choices made.
2. Analyse constraints and opportunities afforded by different programming techniques, frameworks or libraries, and choice of tools in the production of a modern web application.
3. Apply modern industry standards and take into consideration the security, ethical, legality, usefulness, and usability of the finished applications as well as development efficiency.

INSTRUCTIONS

You will continue building upon your work on front-end development as part of the first summative assessment. While the first stage focused on the front-end and user's visual experience, this assessment shifts the priority to the **middleware and backend architecture** - the logic, data management, and security protocols that allow the platform to function at scale.

GitHub and Version Control

You should use the following GitHub Classroom repo for the purpose of the assessment:

<https://classroom.github.com/a/HtBbWeqy>

The Objective

Your primary goal is to transform your static UI into a dynamic, data-driven platform. You must implement a functional server-side solution that handles the "handshake" between the user interface and the persistent data storage. This includes managing user sessions, processing resource submissions (the "Skill Swaps"), and ensuring that the platform remains secure and performant as the user community grows.

Development Expectations

You will create a **Minimum Viable Product (MVP)** that demonstrates a fully integrated stack. This is not merely the "minimum work" required to pass, but a foundational pilot that demonstrates your ability to apply your learnings and evidence you have achieved the learning outcomes of the module.

You will need to provide the technical documentation for your product as outlines in the **Documentation** section.

Minimum Viable Product (MVP)

1. Persistent User Registration and Authentication

- Database Integration: Create a user's collection in MongoDB to store registration details.
- Form Processing: Replace HTML mock forms with Flask routes (@app.route) that handle POST requests.
- Secure Login: Implement a login system where user credentials are verified against the database (passwords must be hashed using a library like Bcrypt or Werkzeug).



2. Dynamic User Profiles

- CRUD Operations: Allow users to create and edit their profile details (Name, Bio, Hobbies).
- Image Management: Implement a functional file upload for profile pictures (storing file paths).

3. Resource Sharing Engine (The "Skill Swap")

- Submission Logic: Create a route for users to post a resource (title, link, description, category).
- Data Retrieval: Dynamically fetch and display all resources from a resources collection instead of using static HTML.
- Categorisation: Ensure resources are filtered by hobby/category via database queries (e.g. `db.resources.find({"category": "Coding"})`).

4. Search and Discovery

- Keyword Search: Implement a functional search bar that queries the MongoDB collection for matches in titles or descriptions.

5. Server-Side Validation and Error Handling

- Data Integrity: Implement Flask-side validation to ensure mandatory fields (*) are present and correctly formatted before being saved to MongoDB.
- Custom Middleware: Create centralised error handlers for 404 (Not Found) or 500 (Server Error) pages to improve the user journey.

6. Interactive Feedback System

- Voting/Rating: Implement a backend logic to track "Upvotes" or "Ratings" for each resource. This requires updating the resource document in the database whenever a user clicks.
- Comments/Reviews: Create a separate comments collection or a nested array within the resource document to store and display user feedback dynamically.

Advanced/Post-MVP Features

7. Resource Bookmarking (Personal Collections)

- User-Resource Mapping: Create a feature allowing users to "Save" a resource. The backend must manage a list of bookmarked resource IDs within the user's database document.

8. AI-Powered Recommendation Section

- AI Tool Filtering: Implement a specific logic-driven section that automatically highlights "AI Tools" based on the resource category or most recent activity.
- Basic Recommendation: Develop an algorithm (e.g. "Most Popular in Your Hobbies") that queries the database to suggest resources to the user based on their profile data.

9. Request and Voting for Skills

- Community Requests: Allow users to "Request" a skill or category they want to learn.
- Voting System: Implement a voting mechanism on these requests to help the users community prioritise which resources should be shared next.

Documentation

Your project repository must contain a comprehensive ReadMe.md file. This document acts as your technical report and is the primary evidence for your research, design decisions, and critical reflections.

1. Project Overview and Stack

- Choice of Tech Stack: Identify your backend framework and database (e.g. Flask & MongoDB).
- Project Goals: Briefly explain the transition from your UI into a functional, data-driven "Community Skill Swap Hub".
- Installation Instructions: Provide a step-by-step guide to setting up the environment (e.g. pip install, setting up the MongoDB URI, and running python app.py).
- Version Control: Provide a link to your GitHub repository and a short description of your branching strategy or commit history.

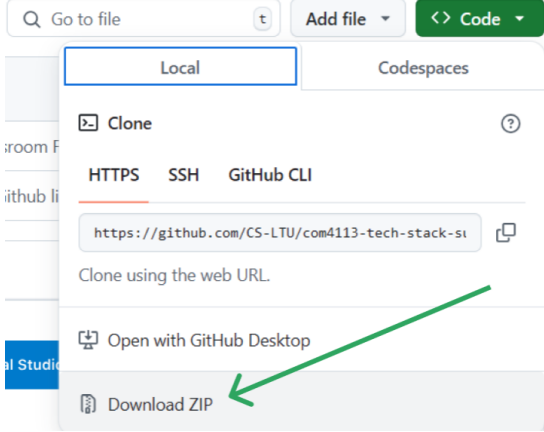
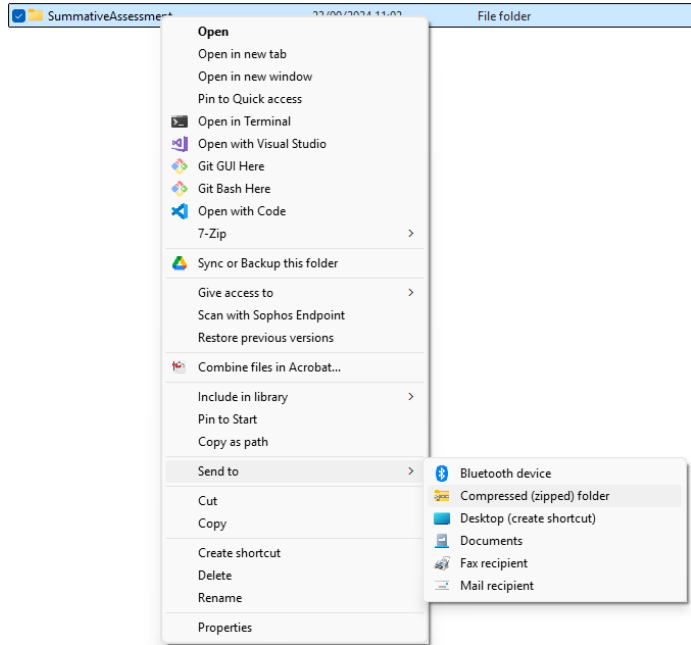


- Basic Legal/Ethical Review: Outline the basic legal and ethical consideration in a community-driven platform.
 - Risk Assessment: Include a list of risks identified in the project, explain the mitigating strategy and evidence if you have implemented any of the mitigating strategies in your project.
2. Technical Architecture and Replication
- Database Schema: Describe your database structure, MongoDB collections or MySQL tables (e.g. users, resources). Explain your rationale for choosing SQL or a NoSQL approach (MySQL or MongoDB).
 - Replication Guide: Include advanced instructions for seeding the database or configuring the connection strings so another developer can replicate your work exactly.
 - Applied Risk Assessment: Identify at least three technical risks (e.g. database connection failure, invalid user input, or data leaks) and describe how your code handles them.
3. Rationale and Software Attributes
- Critical Comparison: Compare the Flask framework with an alternative (e.g. Django or Express). Justify why Flask's micro-framework approach was the most effective choice for this pilot.
 - Good Software Attributes: Discuss how your code adheres to "Separation of Concerns" (e.g. keeping your MongoDB logic separate from your HTML routing).
 - Legal and Ethical Rationale: Critically discuss the legal implications of storing user data (GDPR) and the ethical responsibility of moderating shared links in the Skill Swap Hub.
4. Design, Security, and Critical Reflection
- Provide an ERD (Entity Relationship Diagram) for SQL database or DMD (Data Model Diagram for NoSQL database) that models the structure of your database.
 - Data Protection: Explain how you implemented password hashing and whether this is a legal necessity.
 - Mitigation Evidence: Document at least two specific risk mitigation strategies you implemented (e.g. CSRF protection, input sanitisation, or rate limiting).
 - Reflection on Challenges: Detail the technical hurdles you faced when connecting Flask to database (MongoDB or MySQL Server) and evaluate the effectiveness of your solutions.
 - Referencing: Include a list of at least 10 high-quality resources (academic papers, official documentation, industry blogs) in APA 7th format.
5. In-Code Documentation
- Key Functions: Every Flask route and MongoDB helper function must have a clear comment explaining its purpose and the data it handles.
 - Complex Logic: If you implemented a "Post-MVP" feature (like a rating system or AI recommendation), use comments to guide the reader through the logic flow.
 - Generative AI Disclosure: Include the required statement at the end of your documentation, explicitly choosing either "This assignment used generative AI..." or "This assignment did not use generative AI..."

SUBMISSION DETAILS

RELEASE DATE	23 rd of February 2026
SUBMISSION DATE	Thursday 2 nd of April 2026
DELIVERABLES	You should submit an archive file (.zip) including all your work. If you have synced the latest version of your work to GitHub, you can download an archive format of your project from GitHub.



	 As an alternative, make sure that all of your project files and corresponding documentation is stored in a folder within your machine (laptop/VM/PC) and compress the folder as an .zip file.  Upload the .zip file to Moodle. Only submissions on Moodle are eligible for marking. Before submitting to Moodle, ensure your .zip archive contains: • The Source Code: Complete project folder (excluding node modules or virtual environments). • ReadMe.md: Including tech stack justification, installation guide, replication steps, risks, legal, and ethical considerations. • Project Plan: Your Gantt chart. • GitHub Link: Active URL to the repository in the ReadMe.
SUBMISSION DETAILS	Submit your assignment by uploading it to Moodle <u>before midday</u> on the submission date. This deadline will be automatically and strictly enforced. If your submission is late, your grade may be affected. If you have any issues submitting your work, you must email the assessment team and copy in the module leader <u>before the assessment due time</u>. Do not leave your submission until the last minute to avoid any penalties due to problems with the submission portal. Assessment Team: assessment@leedstrinity.ac.uk Module Leader: yash.b@leedstrinity.ac.uk



We may ask for a demonstration of your work following the submission. If needed, this will be communicated to you individually via email. Please check your emails regularly.

Your feedback / feed forward and mark for this assessment will be provided within 15 working days.

MARKING CRITERIA

Marks are awarded based on the following criteria. Within each part, aim to complete the work for each section before moving on to the next. The following banded marking scheme is used:

<i>Exceptional 1st</i>	100/95/92		<i>2:ii</i>	58/55/52
<i>Outstanding 1st</i>	88/85/82		<i>3rd</i>	48/45/42
<i>1st</i>	78/75/72		<i>Bare Fail</i>	38/35/32
<i>2:i</i>	68/65/62		<i>Fail</i>	25/20/10/0

If you have completed all the preparatory exercises and attended your classes, the estimated additional time required to PASS this assessment is approximately 70 hours.

Note that threshold marking is applied. If you have not met the criteria for one of the requirements in a marking category, your mark will be capped in the previous marking category (max) and you cannot obtain a score in the higher marking categories. Make sure that you have completed all tasks in the 3rd Mark (40%) criteria to pass the module, before focusing on criteria in higher mark categories.

To obtain a 3 rd mark (40%), you must have:	- Delivered the Minimum Viable Product (MVP) with core features functional, specifically User Registration, Profile management, and Resource Posting. - Evidence of a server-side framework (e.g., Express, Flask) handling basic routing. - Produced a ReadMe.md that identifies the backend tech stack and provides basic setup instructions. - Utilised version control (Git) and made project files accessible via a GitHub repository. - Outlined the legal and ethical considerations and provided a generic project risk assessment.
To obtain a 2:ii mark (50%), you must have (in addition to the above):	- Integrated a database (SQL or NoSQL) to manage persistent data for users and resources, successfully replacing all mock data. - Provided a visual project timeline (Gantt Chart) showing the transition from UI to Backend milestones. - Formulated clear instructions in the ReadMe for replicating the development environment and database seeding or migration. - Reflection on Challenges: Detail the technical hurdles you faced when connecting Flask to MongoDB and evaluate the effectiveness of your solutions. - Discussed and provided examples of how your code/prototype adheres to good software attributes. - Compared and justified the chosen backend framework against alternatives, providing a critical technical rationale for its selection.
To obtain a 2:i mark (60%), you must have (in addition to the above):	- Developed custom middleware for session management, and secure authentication (JWT or Passport). - Provided an ERD/DMD that models the structure of your data.



To obtain a 1st mark (70%), you must have (in addition to the above):	○ Implemented at least one "Post-MVP" enhancement (e.g. Resource Bookmarking or a community Rating system) ○ Synthesised information from at least 8 high-quality academic or industry resources using APA 7th referencing. ○ Executed server-side input validation for all mandatory fields to ensure data integrity.
	○ Created and documented at least two optional "Post-MVP" enhancements that involve complex backend logic or external API integrations. ○ Incorporated at least two specific risk mitigation strategies (e.g. Password hashing with salting, CSRF protection, or Rate limiting). ○ Modelled the User Journey visually, mapping front-end interactions directly to back-end API triggers and database updates. ○ Critically reflected on the technical challenges encountered during development and evaluated the effectiveness of the solutions applied. ○ Demonstrated adherence to RESTful API principles for all data manipulation operations (GET, POST, PUT, DELETE). ○ Explained an architect of a deployment-ready solution showcasing cloud-readiness (e.g. Docker and containers, CI/CD pipelines, or Cloud deployment). ○ Innovated beyond the initial brief, demonstrating exceptional technical mastery and original problem-solving.

USE OF GENERATIVE ARTIFICIAL INTELLIGENCE IN THIS MODULE

You may use generative AI such as Microsoft Copilot to assist you in the process of undertaking the assessment in the following ways: brainstorming, research, planning, feedback, editing.

All use of generative AI must be explicitly acknowledged, and any artificially generated content (e.g. images) explicitly labelled, with the source of the AI tool referenced using current APA referencing conventions. You can find further guidance on the library website on their AI webpage).

In submitting your assignment, you agree to disclose the extent to which you have used generative AI in preparing this work and include evidence of your AI use in your appendices (e.g. dated screen shots of your use of this tool or copy and paste your AI chat into Word).

Failure to disclose your generative AI use may result in a zero for your assignment and a referral for academic misconduct (see the Student Academic Misconduct Policy under Essential Info in the MyLTU app).

Include one of the following statements on your assignments.

Either:

This assignment used generative AI in the following ways for the purposes of completing the assignment (choose 1 to 5 of the following): brainstorming, research, planning, feedback, editing.

Or:

This assignment did not use generative AI for the purposes of completing the assignment.

ACADEMIC MISCONDUCT

Academic Misconduct includes all forms of academic dishonesty, whether intentional or accidental, that compromise the integrity of the University's assessment processes. It is essential that you review our [Student Academic Misconduct Policy](#) to understand the guidelines and the serious consequences that may arise if they are not followed.



HELP AND SUPPORT

- Please use the module handbook and the [Computer Science Community Teams site](#) as a source of information. Do try and find the answer out yourself before reaching out for help.
- Support will be provided via Microsoft Teams and email during office working hours. You can also ask questions during your timetabled sessions. You may request a one-to-one meeting with a tutor during their published office hours.
- The Student Support team are available seven days a week to support you in all aspects of student life. This could be for support relating to your course, your accommodation or for more general advice such as relationships or your wellbeing. Log in to the LTU app to access support services.
- The full set of university guidelines on assessments, deadlines, and extensions is available on the LTU app, please familiarise yourself with the documentation.